Persistent

Module 1 – Introduction to Databases

A foundational module for fresh engineering graduates. No prior database experience required — just bring your Java and Docker skills.

What You Will Learn

This module lays the groundwork for everything that follows. By the end of this 45-minute session, you will have a confident understanding of the core ideas behind databases — why they exist, how they're structured, and how developers interact with them.

Data & Databases

What is data, what is a database, and why every modern application depends on one.

DBMS vs RDBMS

Key differences, popular systems like MySQL and PostgreSQL, and where SQL fits in.

Flat File Problems

Why spreadsheets and text files fail at scale — and how databases solve these problems.

ACID & Normalization

Core database guarantees and the basics of organizing tables correctly.

Every modern application — from banking to Netflix — stores and retrieves data inside databases.

Persistent

What is Data?

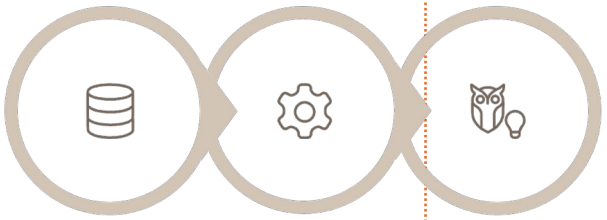
Data is a collection of **raw, unprocessed facts**. On its own, data has no meaning — it only becomes valuable once it is organized, processed, and interpreted into useful information.

Examples of Raw Data

- **Student:** Roll No, Name, Marks
- **Customer:** Name, Mobile, Email, Address
- **Bank:** Account Number, Balance, Transactions

Data → Information

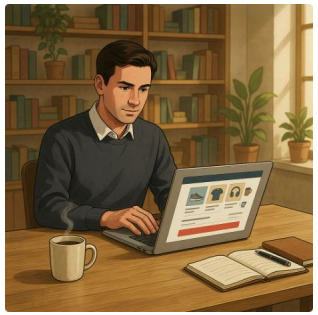
Raw data `101, Rahul, 90` is just numbers and text. Processed, it becomes: *"Rahul scored 90 marks and ranks 1st in the class."*



Persistent

What is a Database?

A database is an **organized collection of related data** that allows efficient storage, retrieval, updating, and management — all at the same time, by multiple users, securely and reliably.



Amazon Example

Customers, Orders, Products, Payments, and Inventory — all stored and linked inside databases, powering billions of transactions daily.

Banking Example

Every account balance, transaction history, and customer profile is managed inside a secure, highly available database system.

Streaming Example

Netflix tracks millions of users, their preferences, watch history, and content metadata — all in real time using distributed databases.

Organized	Secure	Fast Search
Multi-User	Reliable	

Persistent

Why Do We Need Databases?

Imagine maintaining records for 100,000 employees inside a spreadsheet. Searching, updating, and securing that data becomes an operational nightmare. Databases were built to solve exactly this class of problem — at any scale.

Without a Database

- Duplicate records everywhere
- Risk of accidental data loss
- Slow, manual searching
- No access control or security
- Does not scale with growth

✔ With a Database

- Fast, indexed search
- Role-based access security
- Automated backups and recovery
- True multi-user concurrent access
- Scales to millions of records

A database turns a chaotic pile of facts into a structured, queryable, and trustworthy asset.

Persistent

Problems with Flat Files

Flat files — like `.xlsx` or `.csv` — seem convenient for small tasks, but they break down fast in real applications. Here are the six critical failure points every engineer should know.

Duplicate Data

An employee's address is repeated across hundreds of rows. Update one and the rest fall out of sync.

Data Inconsistency

The same customer appears with two different phone numbers in two separate files — which one is correct?

No Relationships

You cannot easily link a customer to their orders. Every join must be done manually, error-prone.

No Security

Anyone with file access can read, edit, or delete any record. There is no concept of roles or permissions.

No Concurrency

Two users editing the same file simultaneously causes corruption or overwriting of each other's changes.

No Recovery

Accidental deletion or a crashed machine means permanent data loss — with no rollback or undo.

DBMS vs RDBMS

A **DBMS** (Database Management System) stores and retrieves data, but treats each file or dataset independently. An **RDBMS** (Relational DBMS) goes further — it stores data in structured, related tables and enforces rules that keep data consistent and secure across the entire system.

Feature	DBMS	RDBMS
Data Storage	Files / Records	Structured Tables
Relationships	Not supported	Fully supported
Normalization	Limited	Fully normalized
Users	Single user	Multi-user concurrent
Security	Basic	Role-based, enterprise-grade
Scalability	Limited	Highly scalable
Examples	File Systems, XML stores	Oracle, MySQL, PostgreSQL

Almost every enterprise application today is built on an RDBMS.

Popular Database Systems

As an engineer, you'll encounter these four systems repeatedly. Each has a different sweet spot — understanding their strengths helps you choose the right tool for the right job.

Oracle Database

Commercial · Enterprise-grade. The go-to choice for banks, telecom companies, and government systems. Known for rock-solid reliability and advanced security features.

MySQL

Open Source · Web-optimized. The most widely used database for web applications. Powers WordPress, Facebook (originally), and thousands of startups worldwide.

SQL Server

Commercial · Microsoft Ecosystem. Deep integration with Azure, .NET, and Windows environments. A top choice in corporate Microsoft-stack applications.

PostgreSQL

Open Source · Feature-rich. Supports JSON, GIS, full-text search, and advanced analytics. Increasingly popular for modern cloud-native applications.

Persistent

Introduction to SQL

SQL — Structured Query Language — is the universal language for communicating with relational databases. Whether you're using MySQL, PostgreSQL, or Oracle, you use SQL to create, read, update, and delete data.

A Simple SQL Query

```
-- Retrieve all student records
SELECT * FROM students;

-- Retrieve top scorers
SELECT name, marks
FROM students
WHERE marks > 85
ORDER BY marks DESC;
```

SQL Command Categories

DDL

CREATE · ALTER · DROP

DML

INSERT · UPDATE · DELETE · SELECT

DCL

GRANT · REVOKE

TCL

COMMIT · ROLLBACK

Persistent

Database Components

Every relational database is built from a small set of fundamental building blocks. Understanding these components is essential before you write your first table or query.

Table

The core unit of storage. A table organizes data into **rows** (records) and **columns** (fields) — similar to a spreadsheet, but with strict structure and rules.

Foreign Key

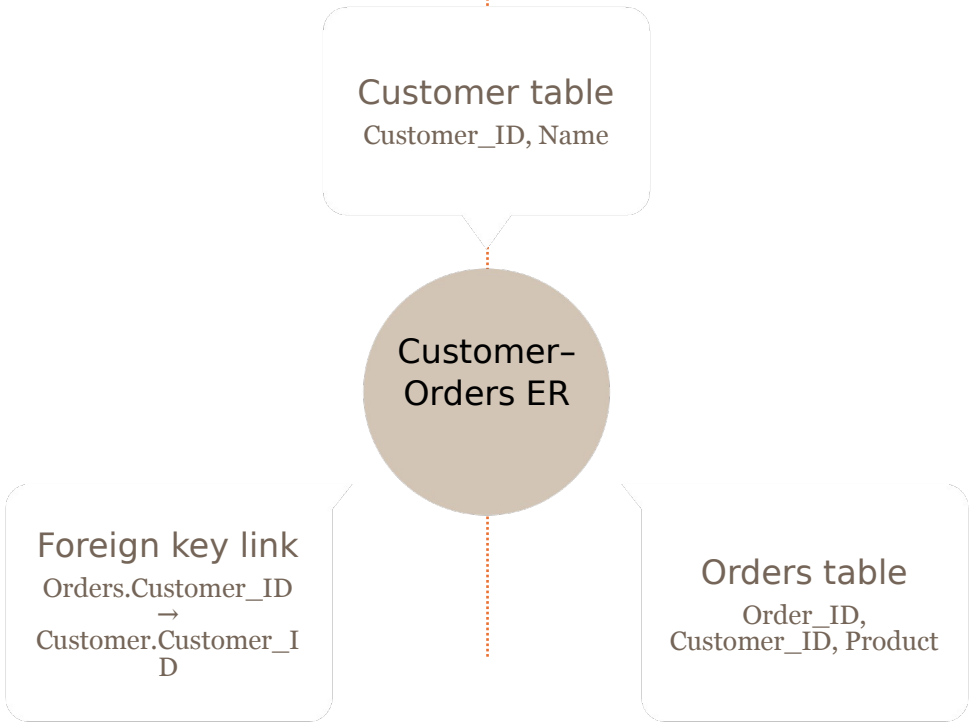
A column in one table that **references the Primary Key** of another. Foreign Keys enforce relationships — e.g., `Customer_ID` in an Orders table links to the Customers table.

Primary Key

A column (or combination) that **uniquely identifies** each row. No two rows can share the same Primary Key value. Example: `Student_ID` in a Students table.

Relationships

Tables relate to each other logically. A classic example: one Customer can have **many Orders** — this is a One-to-Many relationship enforced via Foreign Keys.



Persistent

ACID Properties

ACID is the set of four guarantees that make database transactions **safe and reliable** — even when things go wrong (power failure, network error, concurrent users). Every production-grade RDBMS enforces ACID compliance.

A — Atomicity

A transaction is **all or nothing**. Either every step completes successfully, or the entire transaction is rolled back. No partial updates are ever saved.

C — Consistency

The database always moves from one **valid state to another**. Business rules and constraints are never violated, even mid-transaction.

I — Isolation

Concurrent transactions **do not interfere** with each other. Each transaction executes as if it were the only one running, preventing dirty reads and conflicts.

D — Durability

Once committed, data is **permanently saved** — even if the system crashes immediately after. The database uses write-ahead logs to guarantee this.

ATM Example: When you withdraw cash — money deducted, cash dispensed, balance updated — either ALL three steps succeed, or NONE of them do. That's Atomicity in action.

Introduction to Normalization

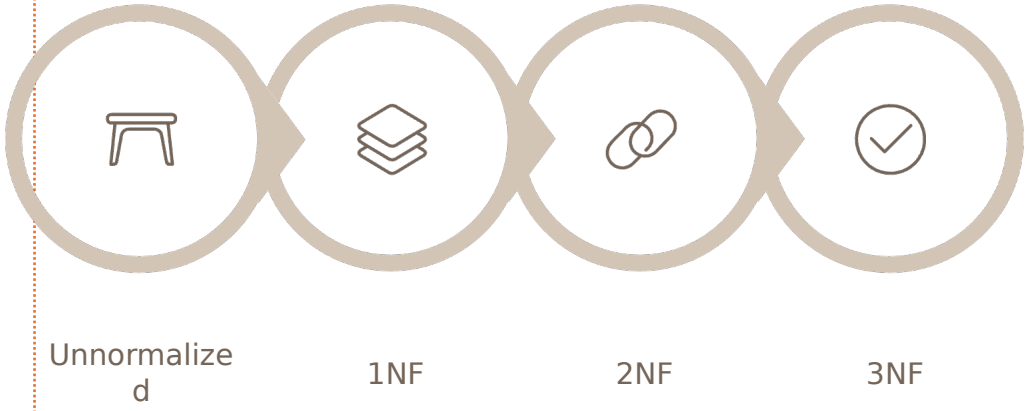
Normalization is the process of **organizing database tables** to eliminate redundant data and improve consistency. A well-normalized database stores each fact exactly once — making updates safer and queries more efficient.

Goals of Normalization

- Eliminate duplicate and redundant data
- Improve storage efficiency
- Maintain data integrity across updates
- Make relationships explicit and clean

Normal Forms — Overview

- **1NF:** No repeating groups; all values must be atomic
- **2NF:** Remove partial dependencies on composite keys
- **3NF:** Remove transitive dependencies between non-key columns



Detailed normalization with worked examples will be covered in a dedicated later module.

Persistent

Module 1 — Summary

You've covered a lot of ground in 45 minutes. Here's everything you now understand — concepts that most developers take months on the job to pick up intuitively.

1

Data vs Information

Raw facts become meaningful only after processing and context.

2

Database Concepts

Organized, secure, multi-user storage — far beyond a spreadsheet.

3

Flat File Problems

Duplicates, inconsistency, no security, no recovery — six real failures.

4

DBMS vs RDBMS

Relational systems add relationships, normalization, and true multi-user support.

5

SQL Foundations

DDL, DML, DCL, TCL — the four pillars of database interaction.

6

ACID & Normalization

Transactions are safe; tables are clean. These guarantees power production systems.

What's Next — Module 2

Now that you understand the *why* behind databases, it's time to get hands-on. In the next module, you'll spin up a real MySQL instance using Docker — the same workflow used on professional engineering teams.

1

Install MySQL via Docker

Pull the official MySQL image and run your first container with a single Docker command.

2

Create Databases & Tables

Use DDL statements to define your first schema — databases, tables, columns, and data types.

3

Write Your First Queries

INSERT, SELECT, UPDATE, DELETE — bring your data to life with real SQL on real data.

Persistent

Key Takeaway

> "A well-designed database is the foundation of every successful software application."

Every system you build as an engineer — APIs, microservices, mobile apps — will rely on a database underneath. The concepts you learned today will follow you throughout your entire career.

Review

Re-read your notes on ACID and Normalization before the next session.

Prepare

Make sure Docker Desktop is installed and running on your machine.

Questions?

Reach out to your instructor or post in the course discussion channel.

Persistent

Thank you